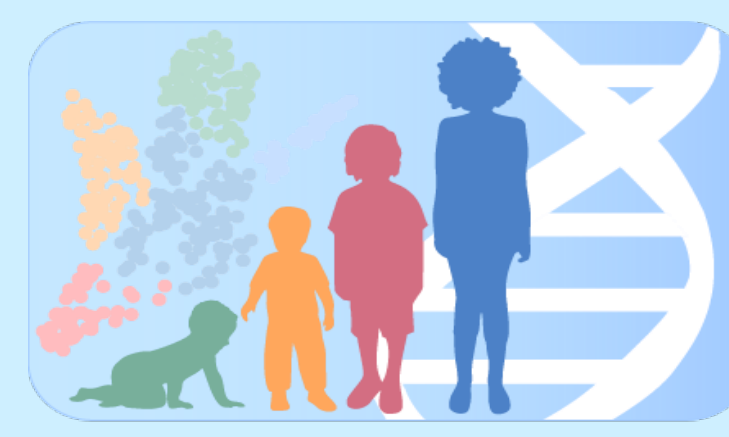




The GTEx Data Ingestion Pipeline

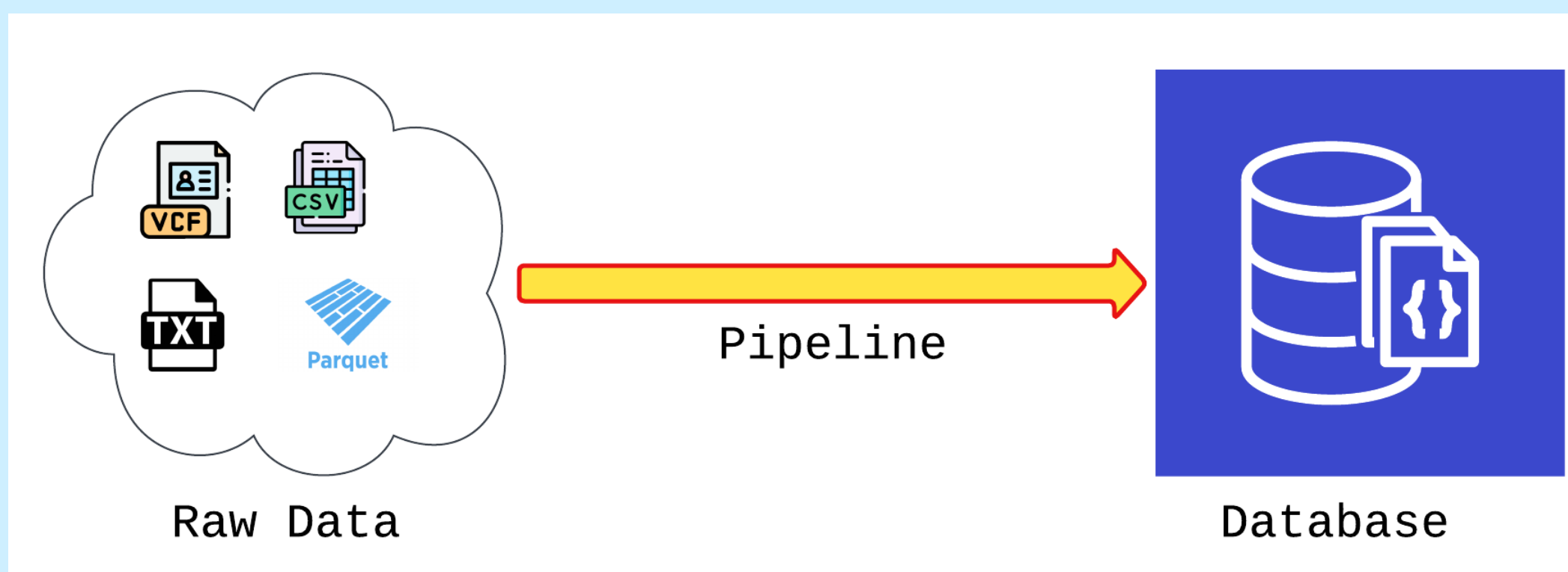
Simple, Scalable, and Extensible

Joseph Okonda, Qiuyue Liu, Jared Nedzel, Kristin Ardlie

The Broad Institute, Cambridge, MA

Background

- Data portals, like GTEx, have revolutionized access to genomic data, **accelerating analysis and discovery**.
- These portals rely on well-defined APIs, which in turn depend on highly structured databases optimized for rapid retrieval.
- However, **raw experimental data typically exists in a less structured form**, scattered across various file formats such as gct, parquet, tsv, and vcf.
- Bridging this gap requires a **robust data ingestion pipeline** that transforms raw data into a structured format suitable for database loading.
- This transformation process is critical, as it represents a **potential point of failure** where errors can be introduced.
- Therefore, the ingestion pipeline needs to be **robust and repeatable**, **simple and scalable**, yet **transparent** and **extensible**.



Main Considerations

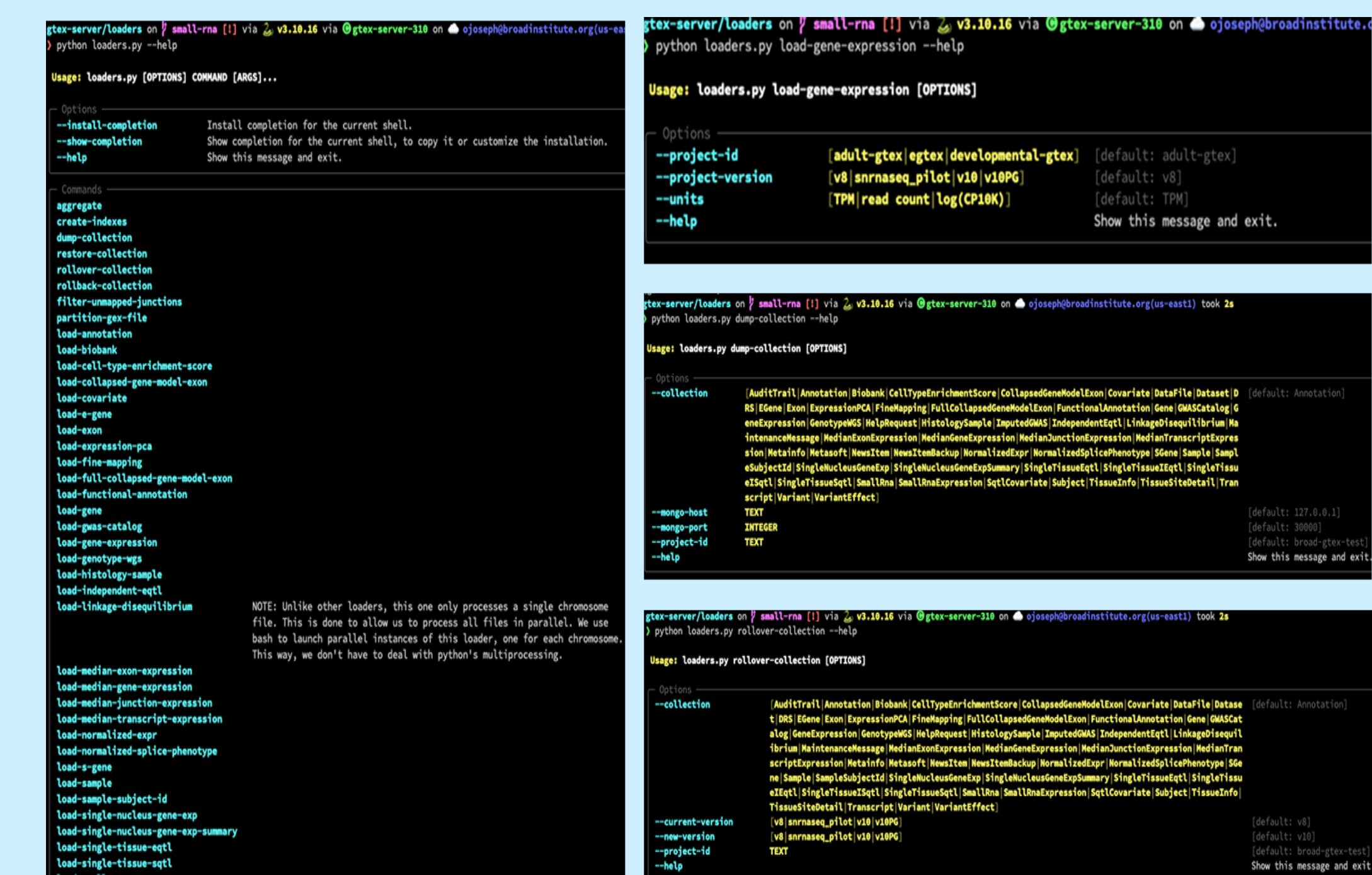
- The pipeline should be **simple**. It should be easy to use, with a clean, intuitive UI, requiring minimal configuration. In particular, it should make input data discovery and management trivial. Further, it should be transparent, providing clear visibility into the entire process, to allow for easy audits.
- The pipeline should be **scalable**. It should gracefully handle input data that are larger than RAM. Loading a 100GB+ dataset should be as easy as loading a 10MB dataset.
- Finally, the pipeline should be **robust, repeatable, and easily extensible**. It should make it easy to debug, and to ingest new data types.

Design Decisions & Tools

- ✓ Maintain a **central, flat repository** for all source files in Google Cloud Storage (GCS), ensuring simple data discovery and management via a YAML manifest.
- ✓ Use a lightweight **abstraction layer** to streamline file handling, **ib** Typer to design a clean CLI, **Pydantic** to ensure structured validation, and **SendGrid** to power automated notifications. Finally, maintain an *append-only audit log* to track all operations for full transparency.
- ✓ For **scalability**, use **Pandas**' *chunked, parallel iteration*, allowing seamless processing of datasets from 10MB to 100GB+. Further, use a modular architecture to enable localized decisions on concurrency and parallelism, ensuring efficient resource use.
- ✓ Maintain strict isolation between production and development envs. In each env, make judicious use of tmp, and staging collections to enable graceful rollback and rollover.

Key Results & Core Lessons

- (R1)**: Transparent nature of the pipeline enables *Faster, and Fearless* data updates.
- (R2)**: Further, we're able to have *Zero-Downtime* data deployments.
- (R3)**: Using Typer gives us a clean, easy to use, *self-documenting CLI*.
- (L1)**: Whenever possible, adopt a design that makes it easy to do parallelism over a python app, instead of inside.
- (L2)**: Carefully model all your input surfaces, and aggressively validate any input type.



The Pipeline

